

Course: MAI511-1 - Advanced Database Technologies

LAB Exercise – IV Normalization

Name: Vismitaa S

Register Number: 2648552

Analysis

Primary Key

(Student_ID, Internship_Title)

Justification: Uniquely identifies each internship record of a student.

Candidate Key

(Student_ID, Internship_Title)

Justification: Can uniquely identify each tuple.

Functional Dependencies

Student_ID → Student_Name, Department

Company_Name → Company_Location

Mentor_Name → Mentor_Designation

Internship_Title → Internship_Status, Company_Name, Mentor_Name

(Student_ID, Internship_Title) → Marks, Grade

Justification: Used to identify partial and transitive dependencies.

Anomalies

Update Anomaly

- If the location of **TCS** changes from Chennai to Coimbatore, every row containing TCS must be updated. Missing one row results in inconsistent data.

Insert Anomaly

- A new company (e.g., Adobe) cannot be added until a student is assigned to an internship.

Delete Anomaly

- Deleting the last student interning at **TCS** also removes all information about TCS and its mentor.

Justification: The unnormalized table stores all details in a single relation, causing high redundancy and all three anomalies.

UNF

QUERY:

```
CREATE TABLE Internship_Details
(
    student_id NUMBER,
    student_name VARCHAR2(30),
    department VARCHAR2(20),
    internship_titles VARCHAR2(100),
    internship_status VARCHAR2(50),
    company_names VARCHAR2(100),
    company_locations VARCHAR2(100),
    mentor_names VARCHAR2(100),
    mentor_designations VARCHAR2(100),
    marks VARCHAR2(50),
    grades VARCHAR2(20)
);
INSERT INTO Internship_Details VALUES
(201, 'Vismitaa', 'AI and ML', 'ML Intern, Data Analyst', 'Pending, Completed',
'TCS, Infosys', 'Chennai, Bangalore', 'Dr. Ramesh, Dr. Priya',
'Assistant Professor, Associate Professor', '95,88', 'A,B');

select * from Internship_Details; |
```

OUTPUT:

STUDENT_ID	STUDENT_NAME	DEPARTMENT	INTERNSHIP_TITLES
201	Vismitaa	AI and ML	ML Intern, Data Analyst

INTERNSHIP_STATUS	COMPANY_NAMES
Pending, Completed	TCS, Infosys

COMPANY_LOCATIONS	MENTOR_NAMES
Chennai, Bangalore	Dr. Ramesh, Dr. Priya

MENTOR_DESIGNATIONS	MARKS	GRADE
Assistant Professor, Associate Professor	95,88	A,B

Justification: The table stores multiple internship details in a single row. Multiple values in the same column violate **1NF**, causing redundancy and anomalies

1NF

QUERY:

```
CREATE TABLE Internship_Details_1NF
(
    student_id NUMBER,
    student_name VARCHAR2(30),
    department VARCHAR2(20),
    internship_title VARCHAR2(30),
    internship_status VARCHAR2(15),
    company_name VARCHAR2(30),
    company_location VARCHAR2(20),
    mentor_name VARCHAR2(30),
    mentor_designation VARCHAR2(30),
    marks NUMBER,
    grade CHAR(1)
);

INSERT ALL
INTO Internship_Details_1NF
VALUES
(201,'Vismिताa','AI and ML','ML Intern','Pending','TCS',
'Chennai','Dr. Ramesh','Assistant Professor',95,'A')
INTO Internship_Details_1NF VALUES
(201,'Vismिताa','AI and ML','Data Analyst','Completed','Infosys',
'Bangalore','Dr. Priya','Associate Professor',88,'B')
SELECT * FROM dual;
select * from internship_details_1nf;
```

OUTPUT:

STUDENT_ID	STUDENT_NAME	DEPARTMENT	INTERNSHIP_TITLE	INTERNSHIP_STAT	COMPANY_NAME
201	Vismिताa	AI and ML	ML Intern	Pending	TCS
201	Vismिताa	AI and ML	Data Analyst	Completed	Infosys

COMPANY_LOCATION	MENTOR_NAME	MENTOR_DESIGNATION	MARKS	G
Chennai	Dr. Ramesh	Assistant Professor	95	A
Bangalore	Dr. Priya	Associate Professor	88	B

JUSTIFICATION: Each column now contains only one value satisfying **1NF**.

2NF

QUERY:

```
CREATE TABLE Students_NF
(
  student_id NUMBER PRIMARY KEY,
  student_name VARCHAR2(30),
  department VARCHAR2(20)
);
INSERT INTO students_nf VALUES(201,'Vismitaa','AI and ML');
CREATE TABLE Internship_NF
(
  internship_id NUMBER PRIMARY KEY,
  internship_title VARCHAR2(30),
  internship_status VARCHAR2(15),
  company_name VARCHAR2(30),
  company_location VARCHAR2(20),
  mentor_name VARCHAR2(30),
  mentor_designation VARCHAR2(30)
);

INSERT ALL
INTO internship_nf VALUES(301,'ML Intern','Pending',
'TCS','Chennai','Dr. Ramesh','Assistant Professor')
INTO internship_nf VALUES(302,'Data Analyst','Completed',
'Infosys','Bangalore','Dr. Priya','Associate Professor')
SELECT * FROM dual;
SELECT * FROM internship_nf;
CREATE TABLE Intern_Evaluation_NF
(
  student_id NUMBER,
  internship_id NUMBER,
  marks NUMBER,
  grade CHAR(1)
);
INSERT ALL
INTO intern_evaluation_nf
VALUES(201,301,95,'A')
INTO intern_evaluation_nf VALUES
(201,302,88,'B')
SELECT * FROM dual;
SELECT * FROM intern_evaluation_nf;
```

OUTPUT:

INTERNSHIP TABLE:

INTERNSHIP_ID	INTERNSHIP_TITLE	INTERNSHIP_STAT	COMPANY_NAME	COMPANY_LOCATION	MENTOR_NAME
301	ML Intern	Pending	TCS	Chennai	Dr. Ramesh
302	Data Analyst	Completed	Infosys	Bangalore	Dr. Priya

MENTOR_DESIGNATION
Assistant Professor
Associate Professor

EVALUATION TABLE:

STUDENT_ID	INTERNSHIP_ID	MARKS	G
201	301	95	A
201	302	88	B

STUDENT TABLE:

STUDENT_ID	STUDENT_NAME	DEPARTMENT
201	Vismitaa	AI and ML

JUSTIFICATION: Student details are stored separately from internship and evaluation details. Partial dependencies are removed.

3NF

QUERY:

```
CREATE TABLE Company_NF
(
  company_id NUMBER PRIMARY KEY,
  company_name VARCHAR2(30),
  location VARCHAR2(20),
  industry_type VARCHAR2(20)
);
INSERT ALL
INTO company_nf VALUES(1, 'TCS', 'Chennai', 'IT')
INTO company_nf VALUES(2, 'Infosys', 'Bangalore', 'IT')
SELECT * FROM dual;
CREATE TABLE Mentor_NF
(
  mentor_id NUMBER PRIMARY KEY,
  mentor_name VARCHAR2(30),
  designation VARCHAR2(30)
);
```

```

INSERT ALL
INTO mentor_nf VALUES(101,'Dr. Ramesh','Assistant Professor')
INTO mentor_nf VALUES(102,'Dr. Priya','Associate Professor')
SELECT * FROM dual;
SELECT * FROM mentor_nf;
CREATE TABLE Internship_3NF
(
internship_id NUMBER PRIMARY KEY,
internship_title VARCHAR2(30),
internship_status VARCHAR2(15),
company_id NUMBER,
mentor_id NUMBER
);
INSERT ALL
INTO internship_3nf VALUES(301,'ML Intern','Pending',1,101)
INTO internship_3nf VALUES(302,'Data Analyst','Completed',2,102)
SELECT * FROM dual;
SELECT * FROM internship_3nf;

```

OUTPUT:

MENTOR TABLE:

MENTOR_ID	MENTOR_NAME	DESIGNATION
101	Dr. Ramesh	Assistant Professor
102	Dr. Priya	Associate Professor

COMPANY TABLE:

COMPANY_ID	COMPANY_NAME	LOCATION	INDUSTRY_TYPE
1	TCS	Chennai	IT
2	Infosys	Bangalore	IT

INTERNSHIP TABLE:

INTERNSHIP_ID	INTERNSHIP_TITLE	INTERNSHIP_STAT	COMPANY_ID	MENTOR_ID
301	ML Intern	Pending	1	101
302	Data Analyst	Completed	2	102

ANAMOLY

UNF

Student: Vismitaa

Internships: ML Intern, Data Analyst

Companies: TCS, Infosys

Mentors: Dr. Ramesh, Dr. Priya

- If **TCS** changes location → update every occurrence (**Update Anomaly**).
- Cannot insert **Google** until a student joins (**Insert Anomaly**).
- Deleting Vismitaa removes the only record of **TCS** and **Dr. Ramesh** (**Delete Anomaly**).

After 3NF

- **Company** table stores TCS only once.
- **Mentor** table stores Dr. Ramesh only once.
- **Students**, **Internship**, and **Intern_Evaluation** are separate.

Therefore:

- Update only one company row.
- Insert a company without adding a student.
- Delete a student without losing company or mentor details.

Parameter	Before Normalization	After Normalization
Number of tables	1	5
Data redundancy	HIGH	LOW
Storage usage	HIGH	LOW
Update anomaly	YES	NO
Insert anomaly	YES	NO
Delete anomaly	YES	NO
Data consistency	HIGH	LOW
Query complexity	LOW	HIGH

JUSTIFICATIONS:

1. Number of Tables

Before: All information is stored in one table.

After: Information is divided into five related tables, improving organization.

2. Data Redundancy

Before: Company, mentor, and student details are repeated in multiple records.

After: Each detail is stored only once, reducing duplication.

3. Storage Usage

Before: More storage is required because duplicate values are stored repeatedly.

After: Storage is optimized since redundant data is removed.

4. Update Anomaly

Before: Updating a company's location requires changing every row where that company appears. Missing one update causes inconsistent data.

After: Company information is stored in one table, so only a single row needs updating.

5. Insert Anomaly

Before: A new company or mentor cannot be added unless there is an internship record for a student.

After: Companies, mentors, students, and internships can all be inserted independently.

6. Delete Anomaly

Before: Deleting the only internship record of a company may also remove all information about that company and its mentor.

After: Deleting a student or internship record does not remove company or mentor information because they are stored separately.

7. Data Consistency

Before: Duplicate values increase the chance of inconsistent information.

After: Since every piece of information is stored only once, the database remains accurate and consistent.

8. Query Complexity

Before: Queries are simple because all data is in one table.

After: Queries require JOIN operations across multiple tables, making them slightly more complex but producing a well-structured and scalable database.